

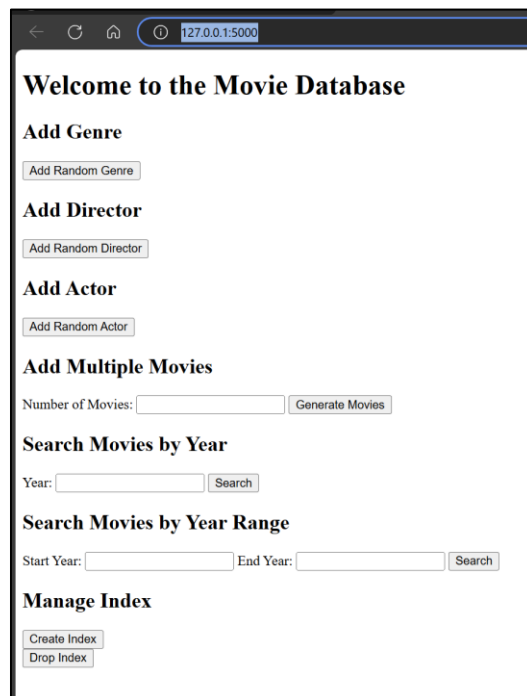
Project: Part 2

This part of the project utilizes what we have created in part 1 and expanded upon it. It will allow us to generate table entries for different tables in the database and then perform searches on the available movies by year. The database is quite large which will better facilitate the exploration of the utilization of indexing.

Our selected database is *new_movies.db* and all SQL queries are working on it.

Each section below will show a screenshot of the application in action along with SQL code. They all access the database and demonstrate operating on said database. Some even have direct SQL queries.

The overall program is a web application that is ran using flask. Here is an image of how the user interface appears.



The screenshot shows a web browser window with the address bar displaying `127.0.0.1:5000`. The page content is as follows:

- Welcome to the Movie Database**
- Add Genre**
 -
- Add Director**
 -
- Add Actor**
 -
- Add Multiple Movies**
 - Number of Movies:
- Search Movies by Year**
 - Year:
- Search Movies by Year Range**
 - Start Year: End Year:
- Manage Index**
 -
 -

1. Add new functions to generate genres, directors, and actors.

Add Genre

Add Random Genre

Successfully added genre: Comedy

```
# db_utils.py

def add_genre(cursor):
    added_genres = set()
    cursor.execute("SELECT GenreName FROM Genres")
    existing_genres = {row[0] for row in cursor.fetchall()}
    new_genre = None

    if len(existing_genres) >= len(genres):
        new_genre = "No more unique genres available"
    else:
        while True:
            genre = generate_genre()[1]
            if genre not in existing_genres and genre not in added_genres:
                cursor.execute("INSERT INTO Genres (GenreName) VALUES (?)",
(genre,))
                added_genres.add(genre)
                existing_genres.add(genre)
                new_genre = genre
                break

    cursor.connection.commit()
    return new_genre
```

```
# app.py

@app.route('/add_genre', methods=['POST'])
def add_genre_route():
    connection = sqlite3.connect('new_movies.db')
    cursor = connection.cursor()
    new_genre = None
    try:
        # Add one unique genre
        new_genre = add_genre(cursor)
        connection.commit()
        if new_genre == "No more unique genres available":
            add_genre_message = "All possible genres have already been added."
        else:
            add_genre_message = f"Successfully added genre: {new_genre}"
    except Exception as e:
        add_genre_message = f"An error occurred while adding genre: {e}"
    finally:
        connection.close()
    return render_template('index.html', add_genre_message=add_genre_message,
genre=new_genre)
```

Directors

Add Director

Add Random Director

Successfully added director: Quest Ibarra, 1928-06-02

```
# db_utils.py

# Function to add directors to the database
def add_director(cursor):
    person = generate_person()
    cursor.execute("INSERT INTO Directors (DirectorName, BirthDate) VALUES (?, ?)",
                   (person["name"], person["birthday"]))
    cursor.connection.commit()
    director_info = f"{person['name']}, {person['birthday']}"
    return director_info
```

```
# app.py

@app.route('/add_director', methods=['POST'])
def add_director_route():
    connection = sqlite3.connect('new_movies.db')
    cursor = connection.cursor()
    new_director = None
    try:
        # Check the current number of directors
        cursor.execute("SELECT COUNT(*) FROM Directors")
        count = cursor.fetchone()[0]

        if count >= 25:
            add_director_message = "Cannot add more directors. The limit of 25 directors has been reached."
        else:
            new_director = add_director(cursor)
            add_director_message = f"Successfully added director: {new_director}"
    except Exception as e:
        add_director_message = f"An error occurred while adding director: {e}"
    finally:
        connection.close()
        print(add_director_message)
    return render_template('index.html', add_director_message=add_director_message, director=new_director)
```

Actors

Add Actor

Add Random Actor

Successfully added actor: Evan Quimby, 1965-02-10

```
# db_utils.py

# Function to add actors to the database
def add_actor(cursor):
    person = generate_person()
    cursor.execute("INSERT INTO Actors (ActorName, BirthDate) VALUES (?, ?)",
                  (person["name"], person["birthday"]))
    cursor.connection.commit()
    actor_info = f'{person["name"]}, {person["birthday"]}'
    return actor_info

# app.py

@app.route('/add_actor', methods=['POST'])
def add_actor_route():
    connection = sqlite3.connect('new_movies.db')
    cursor = connection.cursor()
    new_actor = None
    try:
        # Check the current number of actors
        cursor.execute("SELECT COUNT(*) FROM Actors")
        count = cursor.fetchone()[0]

        if count >= 50: # Assuming we want to limit to 50 actors for example
            add_actor_message = "Cannot add more actors. The limit of 50 actors has been reached."
        else:
            new_actor = add_actor(cursor)
            add_actor_message = f'Successfully added actor: {new_actor}'
    except Exception as e:
        add_actor_message = f'An error occurred while adding actor: {e}'
    finally:
        connection.close()
        print(add_actor_message)
    return render_template('index.html', add_actor_message=add_actor_message, actor=new_actor)
```

2. Add a new function to generate movies.

Add Multiple Movies

Number of Movies:

Successfully added 100 movie(s)!

```
# db_utils.py

# Function to add movies to the database, update MovieGenres, and add actors to
MovieActors
def add_movie(cursor, genre_id, director_id):
    movie = generate_movie(genre_id, director_id)
    title, year, rating, description, director_id = movie

    # Insert into Movies table
    cursor.execute("INSERT INTO Movies (Title, ReleaseYear, Rating, Description,
DirectorID) VALUES (?, ?, ?, ?, ?)",
                    (title, year, rating, description, director_id))

    # Get the last inserted MovieID
    movie_id = cursor.lastrowid
    return movie_id
```

```
# db_utils.py

def add_movie_genre(cursor, movie_id, genre_id):
    cursor.execute("INSERT INTO MovieGenres (MovieID, GenreID) VALUES (?, ?)",
(movie_id, genre_id))

def add_movie_actor(cursor, movie_id, actor_id):
    cursor.execute("INSERT INTO MovieActors (MovieID, ActorID) VALUES (?, ?)",
(movie_id, actor_id))
```

```
# db_utils.py

def get_table_length(cursor, table_name):
    cursor.execute(f"SELECT COUNT(*) FROM {table_name}")
    return cursor.fetchone()[0]
```

```

# app.py

@app.route('/add_movies', methods=['POST'])
def add_movie_route():
    connection = sqlite3.connect('new_movies.db')
    cursor = connection.cursor()
    try:
        # Get form data
        number_of_movies = int(request.form['num_movies'])

        len_genres = get_table_length(cursor, "Genres")
        len_directors = get_table_length(cursor, "Directors")
        len_actors = get_table_length(cursor, "Actors")

        if len_genres == 0:
            add_movie_message = "Please add genres before adding movies."
        elif len_directors == 0:
            add_movie_message = "Please add directors before adding movies."
        elif len_actors == 0:
            add_movie_message = "Please add actors before adding movies."
        else:
            # Loop for number_of_movies
            for _ in range(number_of_movies):
                cursor.execute("SELECT GenreID FROM Genres")
                genres = cursor.fetchall()
                genre_id = random.choice(genres)[0]

                cursor.execute("SELECT DirectorID FROM Directors")
                directors = cursor.fetchall()
                director_id = random.choice(directors)[0]

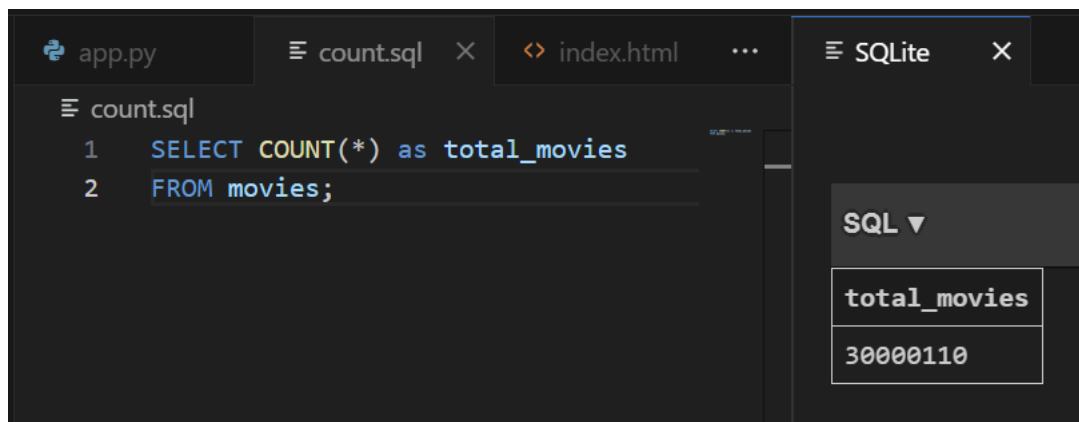
                cursor.execute("SELECT ActorID FROM Actors")
                actors = cursor.fetchall()
                actor_id = random.choice(actors)[0]

                movie_id = add_movie(cursor, genre_id, director_id)
                add_movie_genre(cursor, movie_id, genre_id)
                add_movie_actor(cursor, movie_id, actor_id)

            connection.commit()
            add_movie_message = f"Successfully added {number_of_movies} movie(s)!"
    except Exception as e:
        add_movie_message = f"An error occurred while adding movies: {str(e)}"
        print(f"Error in add_movie_route: {str(e)}")
    finally:
        connection.close()
    return render_template('index.html', add_movie_message=add_movie_message)

```

3. Generate adequate number of movies and execute searches with and without index.



Manage Index

Create Index

Drop Index

Index dropped successfully!

```
# search_movies.py

def drop_index(cursor):
    cursor.execute("DROP INDEX IF EXISTS idx_year")
```

a) Search Movies by Year (without index)

Search Movies by Year

Year:

Number of movies found: 344517

Execution time before indexing: 2.209 seconds.

```
# search_movies.py

def count_movies_by_year(cursor, year):
    start_time = time.time()
    cursor.execute("SELECT COUNT(*) FROM Movies WHERE ReleaseYear=?", (year,))
    count = cursor.fetchone()[0]
    end_time = time.time()
    execution_time = end_time - start_time
    return count, execution_time
```

```
# app.py

@app.route('/search_by_year', methods=['POST'])
def search_by_year():
    year = int(request.form['year'])
    connection = sqlite3.connect('new_movies.db')
    cursor = connection.cursor()
    try:
        count, exec_time = count_movies_by_year(cursor, year)
        search_year_message = (f"Number of movies found: {count} <br>"
                               f"Execution time before indexing: {exec_time:.3f}"
                               "seconds.")
    except Exception as e:
        count = 0
        exec_time = 0
        search_year_message = f"An error occurred while searching by year: {e}"
    finally:
        connection.close()
    return render_template('index.html', search_year_message=search_year_message)
```


b) Search movies by range (without index)

Search Movies by Year Range

Start Year: End Year:

Number of movies found from 1999 to 2024: 8961823

Execution time with indexing: 2.283 seconds.

```
# search_movies.py

def count_movies_by_year_range(cursor, start_year, end_year):
    start_time = time.time()
    cursor.execute("SELECT COUNT(*) FROM Movies WHERE ReleaseYear BETWEEN ? AND ?",
    (start_year, end_year))
    count = cursor.fetchone()[0]
    end_time = time.time()
    execution_time = end_time - start_time
    return count, execution_time
```

```
# app.py

@app.route('/search_by_year_range', methods=['POST'])
def search_by_year_range():
    start_year = int(request.form['start_year'])
    end_year = int(request.form['end_year'])
    connection = sqlite3.connect('new_movies.db')
    cursor = connection.cursor()
    try:
        # Measure execution time with the index
        count, exec_time = count_movies_by_year_range(cursor, start_year, end_year)

        # Prepare the message
        search_year_range_message = (f"Number of movies found from {start_year} to
{end_year}: {count} <br>"
                                     f"Execution time with indexing:
{exec_time:.3f} seconds.")
    except Exception as e:
        count = 0
        exec_time = 0
        search_year_range_message = f"An error occurred while searching by year
range: {e}"
    finally:
        connection.close()
    return render_template('index.html',
search_year_range_message=search_year_range_message)
```

c) Do both searches with index

Manage Index

Create Index

Drop Index

Index created successfully!

```
# search_movies.py

def create_index(cursor):
    cursor.execute("CREATE INDEX idx_year ON Movies(ReleaseYear)")
```

Search Movies by Year

Year:

Number of movies found: 344517

Execution time before indexing: 0.013 seconds.

Search Movies by Year Range

Start Year: End Year:

Number of movies found from 1999 to 2024: 8961823

Execution time with indexing: 0.282 seconds.

Conclusion

SQL Query	Before Index	After Index
Search Movies by Year	2.209 sec	0.013 sec
Search Movies by Year Range	2.283 sec	0.282 sec

Indexing has a significant impact on performance. The index reduced search times drastically. It is important to note that selection of a key is an important piece of the puzzle and will effect the efficiency of the indexed search.